WEEK 1 · LESSON 6 OF 24 · HANDOUT

# Week 1 Project: Read Handwritten Digits

Study time: **about 90 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=tIeHLnjs5U8 — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Neural networks and backpropagation, step by step (building micrograd) — https://www.youtube.com/watch?v=VMj-3S1tku0

🎓 **Go deeper:** freeCodeCamp: PyTorch for Deep Learning & Machine Learning — full course — https://www.youtube.com/watch?v=V_xro1bcAuA

**Your goal in this lesson:** Build, train and evaluate a neural network that reads handwritten digits — first from scratch in NumPy, then with PyTorch autograd — and explain its mistakes.

## The brief

Post offices, banks and schools still read handwritten numbers on forms. Your task: build a network that reads digits 0–9 from small images, using the **digits** dataset built into scikit-learn (1,797 images of 8 × 8 pixels, from 43 writers).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X = digits.data / 16.0            # pixel values 0..16 -> 0..1
y = digits.target
print(X.shape, np.bincount(y))

fig, axes = plt.subplots(1, 8, figsize=(10, 2))
for ax, img, label in zip(axes, digits.images, y):
    ax.imshow(img, cmap="gray_r")
    ax.set_title(label)
    ax.axis("off")
plt.show()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)
```

Each image is flattened into **64 numbers**. Dividing by 16 puts every pixel between 0 and 1 — networks learn faster with small, similar-sized inputs.

## Part 1 — A network from scratch (NumPy)

Architecture: 64 inputs → 32 ReLU neurons → 10 outputs with softmax, trained with categorical cross-entropy.

```
rng = np.random.default_rng(0)
W1 = rng.normal(0, np.sqrt(2 / 64), (64, 32)); b1 = np.zeros(32)
W2 = rng.normal(0, np.sqrt(1 / 32), (32, 10)); b2 = np.zeros(10)
Y = np.eye(10)[y_train]                       # one-hot labels

def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)

lr, batch = 0.1, 64
for epoch in range(60):
    order = rng.permutation(len(X_train))
    for start in range(0, len(order), batch):
        idx = order[start:start + batch]
        xb, yb = X_train[idx], Y[idx]
        z1 = xb @ W1 + b1; a1 = np.maximum(0, z1)
        p = softmax(a1 @ W2 + b2)
        dz2 = (p - yb) / len(idx)             # softmax + cross-entropy gradient
        dW2 = a1.T @ dz2; db2 = dz2.sum(0)
        dz1 = (dz2 @ W2.T) * (z1 > 0)
        dW1 = xb.T @ dz1; db1 = dz1.sum(0)
        W1 -= lr * dW1; b1 -= lr * db1; W2 -= lr * dW2; b2 -= lr * db2

def predict(Xs):
    return (np.maximum(0, Xs @ W1 + b1) @ W2 + b2).argmax(axis=1)

print("NumPy network test accuracy:", round((predict(X_test) == y_test).mean(), 3))
```

The softmax-plus-cross-entropy gradient is simply **prediction − one-hot truth** — the same pattern as for one sigmoid neuron. You should reach about **0.97** accuracy.

## Part 2 — The same network with autograd

```
import torch
import torch.nn.functional as F

torch.manual_seed(0)
Xtr = torch.tensor(X_train, dtype=torch.float32); ytr = torch.tensor(y_train)
Xte = torch.tensor(X_test, dtype=torch.float32); yte = torch.tensor(y_test)

W1t = (torch.randn(64, 32) * (2 / 64) ** 0.5).requires_grad_()
b1t = torch.zeros(32, requires_grad=True)
W2t = (torch.randn(32, 10) * (1 / 32) ** 0.5).requires_grad_()
b2t = torch.zeros(10, requires_grad=True)
params = [W1t, b1t, W2t, b2t]

for epoch in range(60):
    perm = torch.randperm(len(Xtr))
    for start in range(0, len(perm), 64):
        idx = perm[start:start + 64]
        logits = torch.relu(Xtr[idx] @ W1t + b1t) @ W2t + b2t
        loss = F.cross_entropy(logits, ytr[idx])      # softmax + cross-entropy
        loss.backward()
        with torch.no_grad():
            for p in params:
                p -= 0.1 * p.grad
                p.grad.zero_()

with torch.no_grad():
    pred_t = (torch.relu(Xte @ W1t + b1t) @ W2t + b2t).argmax(dim=1)
print("PyTorch network test accuracy:", round((pred_t == yte).float().mean().item(), 3))
```

`F.cross_entropy` takes raw logits and integer labels and applies softmax inside. Both versions should land close to each other.

## Part 3 — Evaluate like a professional

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

pred = predict(X_test)
ConfusionMatrixDisplay(confusion_matrix(y_test, pred)).plot(cmap="Blues")
plt.show()

wrong = np.where(pred != y_test)[0]
print("mistakes:", len(wrong), "of", len(y_test))
fig, axes = plt.subplots(1, min(6, len(wrong)), figsize=(10, 2))
for ax, i in zip(np.atleast_1d(axes), wrong[:6]):
    ax.imshow(X_test[i].reshape(8, 8), cmap="gray_r")
    ax.set_title(f"true {y_test[i]}, said {pred[i]}")
    ax.axis("off")
plt.show()
```

Look at the mistakes. Are they digits a person would also find hard?

## Part 4 — Compare with a baseline

Train `LogisticRegression(max_iter=2000)` from scikit-learn on the same split. A network should beat it — but by how much? On tiny 8 × 8 images the gap is small; on real photos it becomes enormous.

## Part 5 — Report (one page)

1. Data: size, image size, classes, how you scaled the pixels.
2. Architecture and training settings (layers, learning rate, batch size, epochs).
3. Test accuracy of both versions and of the baseline.
4. The confusion matrix and the three most common confusions, with a short explanation.
5. One idea to improve the model, and one real use in Rwanda (for example reading marks on exam sheets).

## Marking guide (self-check)

| Area | Excellent work |
|---|---|
| Data | Pixels scaled, stratified split, sample images shown |
| From scratch | NumPy network trains and reaches about 0.97 |
| Autograd | PyTorch version matches the NumPy result |
| Evaluation | Confusion matrix and mistakes analysed |
| Communication | Clear report with a baseline comparison |

**Remember this:** You built the same digit reader twice — by hand and with autograd — and now understand exactly what deep-learning libraries automate.

Name: _____ Date: _____

## Activity

### Task 1

Complete Parts 1–5. Then add a second hidden layer (64 → 64 → 32 → 10) to the PyTorch version. Does it help on this small dataset?

## Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why divide the digit pixels by 16?
   - A. To put every input between 0 and 1, which helps training
   - B. To make images bigger
   - C. To change the labels
   - D. To delete them
2. How many inputs does the network take for one 8 × 8 image?
   - A. 8
   - B. 16
   - C. 10
   - D. 64
3. What is the gradient of softmax + cross-entropy with respect to the logits?
   - A. The learning rate
   - B. Always zero
   - C. Predicted probabilities minus the one-hot truth
   - D. The input pixels
4. What does F.cross_entropy expect?
   - A. Probabilities and one-hot labels
   - B. Images
   - C. Raw logits and integer class labels
   - D. Only one class
5. Why compare the network with logistic regression?
   - A. To make the network slower
   - B. Because networks cannot be evaluated
   - C. To show whether the extra complexity is worth it
   - D. It is required by PyTorch
6. What is a mini-batch in this project?
   - A. 64 random training images used for one update
   - B. The test set
   - C. All 1,797 images
   - D. One pixel

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.